

实验 5：正向运动学

概述

在本实验中，我们将分别使用 PoE 和 D-H 参数法实现正向运动学。具体来说，我们将为你提供 RX150 机械臂的原理图，你需要使用两种方法（PoE 和 D-H）分别补充完整两个函数（Python 程序）。每个函数的输入参数应该是关节角度，返回值应该是末端执行器的位置（为了简化计算，我们忽略了末端的朝向）。

你可以在 Gazebo 中生成一个 RX150 机械臂，输入一些随机关节角度，看看末端执行器的位置。你可以将这些值与你的函数计算出的结果进行比较，他们的结果应该是相同的。我们会使用一些测试案例对你的正向运动学函数进行测试，以便评分。

作业提交

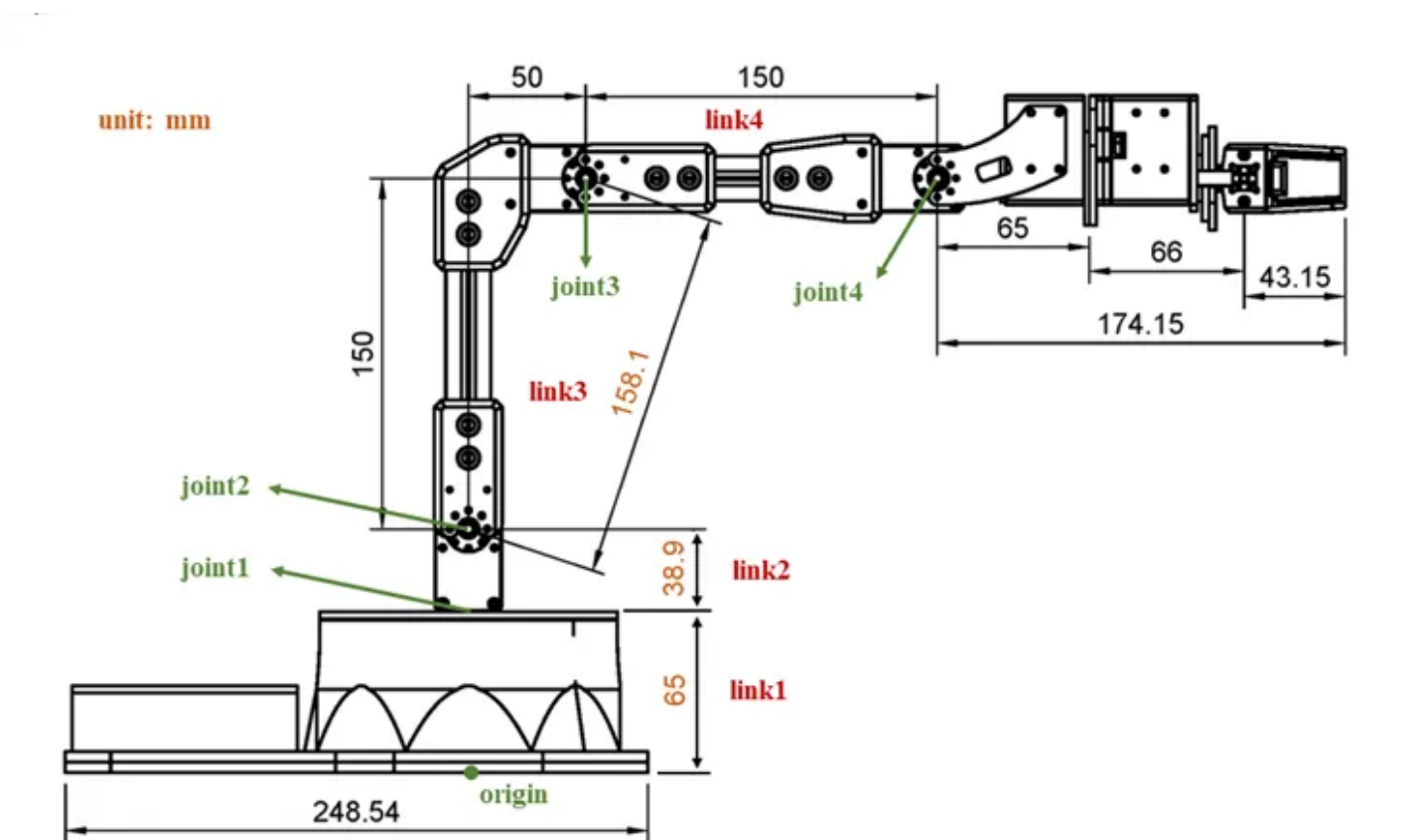
- 需要提交的文件：
 - `forward_kinematics_poe.py`
 - `forward_kinematics_dh.py`
 - 以 PDF 文件格式提交实验报告，包含实验成功的截图并简要叙述实现过程
- 评分标准：
 - 我们将通过几个测试案例对脚本进行测试，并根据正向运动学结果的准确性给出分数。X、Y、Z 轴的绝对误差之和不应超过 0.002 米。

编程提示

- 我们为你提供了两个 Python 脚本 `forward_kinematics.py` 和 `test_forward_kinematics.py`，但你只需完成并提交第一个脚本。第二个脚本仅用于测试。请把第一个脚本复制两次，分别命名为 `forward_kinematics_poe.py` 和 `forward_kinematics_dh.py`，并在两个脚本中分别实现 PoE 和 D-H 方法。对于 PoE 方法，你可以选择 space frame 或者 body frame 中的一种。
- 为了简化计算，我们将把 `joint 4` 视为末端执行器。换句话说，你需要返回 `joint 4` 的位置，而不是实际的 RX150 的末端执行器。下面的机械臂原理图中有详细描述，你可以从中找到哪个是 `joint 4`。
- 你可以使用第三方库中的函数来简化计算。例如，《现代机器人学》的教科书的作者在这个库中提供了一些数学工具：<https://github.com/NxRLLab/ModernRobotics>。
- 请注意计算中使用的数据类型。

- 如果两个矩阵 `A` 和 `B` 都是 `np.array` 类型，那么 `A * B` 将**不执行**矩阵乘法，而是执行对应元素间的乘法。
- 如果两个矩阵 `A` 和 `B` 都是 `np.array` 类型，那么 `np.dot(A, B)` 将**执行**矩阵乘法。
- 如果两个矩阵 `A` 和 `B` 的类型是 `np.matrix`，那么 `A * B` 将**执行**矩阵乘法。
- `numpy` 和 `modern_robotics` 库中一些可能会有帮助的函数。
 - `np.cross(A, B)` 两个向量的叉乘
 - `np.concatenate([A, B])` 两个向量的顺序连接
 - `mr.VecTose3(S)` 将 6x1 twist vector `S` 转换为 `se(3)` 中的 4x4 矩阵
 - `mr.MatrixExp6(M)` 对 `se(3)` 中的 4x4 矩阵进行矩阵指数运算

RX150 机械臂原理图



在仿真中测试 RX 150 机械臂

- 我们建议先启动机器人，用GUI来设定关节角度，来熟悉机械臂的各关节结构和正方向。(这些机械臂的软件包应该早在实验 1 中就已经安装完整了。)

代码块

```
1  roslaunch interbotix_xsarm_descriptions xsarm_description.launch.py
   robot_model:=rx150 use_joint_pub_gui:=true
```

- 接着，你可以关闭GUI，然后完成你自己的程序，并尝试用程序去控制机械臂。（不关闭GUI的话，会出现GUI和你的程序同时给机械臂发指令的情况，机械臂会混乱不知道该听谁的。）

代码块

```
1 python3 test_forward_kinematics.py
```

- 要检查末端执行器的实际位置，请在另一个终端运行以下命令。

代码块

```
1 ros2 run tf2_ros tf2_echo rx150/base_link rx150/wrist_link
```

- 可以把程序计算出来的结果和机械臂的实际位置做一个对比，这两者应该是一致的。